

Reading the role, not the keywords

Intelligent Candidate Discovery & Ranking

An interpretable, recruiter-grade ranker for the Senior AI Engineer (Founding Team) role — fully offline, CPU-only, ~25s over 100,000 profiles.

0 honeypots
in top 100

100% relevant
titles in top 100

96% India ·
51% preferred city

~25s / 100K ·
no GPU · no network

The problem is a reading-comprehension problem

The JD is written to defeat keyword matchers. It explicitly says: *“the right answer is NOT to find candidates whose skills section contains the most AI keywords — that’s a trap we built into the dataset.”* So the task isn’t search; it’s reasoning about the gap between what the JD **says** and what it **means**.

What the JD literally lists

- Embeddings retrieval, vector DBs, hybrid search
- Strong Python, ranking-eval frameworks
- 5–9 years; LLM fine-tuning (nice-to-have)

What the JD actually means

- Has **shipped** ranking/search/recsys to real users at a product company
- Tilts shipper over researcher; writes production code now
- Reachable: active, responsive, in/near Pune–Noida
- NOT: keyword-stuffers, consulting-only, title-chasers, CV/speech-only

Design consequence: title/career evidence and the *meaning* of the work must outweigh the skills array — and plausibility + availability must gate everything.

Architecture — a transparent funnel

100,000 candidates	stream-load, one pass
Permissive prefilter	drop obvious non-fits cheaply; never lose a plausible match
TF-IDF semantic match	career text vs a structured role query — meaning, not keywords
Interpretable score	5 components × availability × trap penalty
Top-100 + reasoning	spec tie-break, grounded explanations, CSV

Why TF-IDF and not a neural embedding?

The constraints forbid network and GPU at ranking time and cap runtime at 5 min on CPU. TF-IDF over career text captures meaning beyond exact keyword overlap (a candidate who “built a recommendation system at a product company” matches the role query without ever writing “RAG”), needs zero downloads, and runs in milliseconds. Meaning is one signal among many — title/career and career-text evidence dominate the blend, which is what actually defeats keyword stuffing.

The score — five readable components, two multipliers

Component	Wt	What it reads	Why it matters
Title / career fit	0.30	current title + best/recent roles, duration-weighted	Decisive guard vs keyword-stuffers — a 'Marketing Manager' with AI skills still scores ~0
Semantic meaning	0.24	TF-IDF cosine vs role query	Surfaces plain-language Tier-5s who did the work without the buzzwords
Career-text evidence	0.18	free-text of every role + summary	Reads the work; catches quiet anti-signals ('lighter weight than ranking')
Trust-weighted skills	0.16	endorsements × usage × proficiency	Un-stuffable: a list of 'expert' skills with no usage/endorsements counts for almost nothing
Experience band	0.12	soft taper around 5–9 (ideal 6–8)	A range, not a cutoff — strong outliers still rank, with an honest concern noted
× Availability	mult	recency, response rate, open-to-work, demand	'Perfect on paper but dormant / 5% response' → not actually hireable → down-weighted
× Trap penalty	mult	honeypot, consulting-only, hopper, CV/speech-only, LLM-wrapper	Encodes the JD's explicit 'do NOT want' list; honeypots are forced to ~0

$$\text{fit} = (0.30 \cdot \text{title/career} + 0.24 \cdot \text{meaning} + 0.18 \cdot \text{text-evidence} + 0.16 \cdot \text{skills} + 0.12 \cdot \text{experience} + \text{location}) \times \text{availability} \times \text{trap}$$

Every trap in the dataset, and how it's neutralized

Trap	Mechanism that handles it
Keyword-stuffers (AI skills, wrong title)	Title/career fit dominates; skills are trust-weighted, not counted
Honeypots (~80, 'subtly impossible')	Internal-consistency checks (job/skill tenure > whole career; expert+0 months) → forced down
Plain-language Tier-5 (real work, no buzzwords)	TF-IDF meaning match + career-text evidence surface them
Title-chasers / job-hoppers	Penalty for multiple sub-18-month stints
Consulting-only careers	Penalty, with the JD's 'OK if prior product experience' carve-out
CV / speech / robotics without NLP/IR	Off-domain skill drag + penalty when no NLP/IR appears in the text
Dormant 'perfect on paper'	Availability multiplier: recency × response rate × open-to-work

Honeypot detection optimizes for **precision** (wide tolerances) — a false positive would knock out a real candidate, and the JD reasoning already buries the keyword-perfect honeypots via title/career fit.

Results & the reasoning layer

Top-100 audit (offline, no ground truth)

- **0** honeypots in top 100 (Stage-3 gate: <10%)
- **100/100** in ML/AI/search/recsys title families
- **96/100** in India · **51/100** in preferred cities
- Scores strictly non-increasing; format validator: PASS
- Runtime ~25s for 100K on CPU, no network

Reasoning is grounded, varied, rank-consistent

Every clause comes only from the candidate's own record — no hallucinated skills. Tone tracks the score, and phrasing is deterministically varied (mean pairwise similarity 0.23), so Stage-4 review sees specific, honest, non-templated text.

Rank 1 — 0.91

“Strong fit: Staff Machine Learning Engineer, 7.0 yrs — depth in Semantic Search, pgvector, Pinecone, BM25; production search/recsys at scale; open to relocating to Pune/Noida.”

Rank 6 — 0.81 (honest concern surfaced)

“Strong fit: Senior Applied Scientist, 16.2 yrs — track record of applied retrieval and ranking... Concern: experience 16.2y above the 5–9 band.”

Reproduce

```
python rank.py --candidates ./data/candidates.jsonl --out ./outputs/submission.csv
```

Deterministic · CPU-only · offline · validated · sandbox demo via app.py. Full design rationale lives in the code (src/role_intent.py reads the JD line by line).